

SUPPLY CHAIN ROUTING ENGINE

Distributed Logistics Platform

Optimize • Route • Fulfill • Analyze

Team Members

Member 1

Core & UI Foundation

Core Algorithms, Database &
Frontend Foundation Lead

Member 2

Backend & Checkout

API, Logistics Orchestration
& Checkout Lead

Member 3

Frontend & Analytics

Frontend, Analytics &
Geospatial Visualization Lead

Problem Statement & Objectives

PROBLEM

Traditional e-commerce systems hide logistics complexity — hosts have no visibility into routing decisions, warehouse selection, or inventory allocation.

OBJECTIVES

- ❖ Real-time Control Tower for logistics operators (Host View)
- ❖ Seamless e-commerce shopping experience (Customer View)
- ❖ Multi-warehouse routing algorithm with distance optimization
- ❖ Redis distributed locks preventing overselling under concurrent load
- ❖ Gemini AI plain-language routing explanations
- ❖ Shared backend — both views use the same database & routing engine

Technology Stack

React 19 + Vite

FRONTEND

Component-driven SPA with hot reload

Node.js + Express

API SERVER

RESTful endpoints, middleware chain

PostgreSQL

DATABASE

ACID transactions, relational integrity

Redis

CACHE / LOCKS

Per-SKU distributed locks, rate limiting

Gemini API

AI ENGINE

Async explanation generation, fallback

MapLibre GL

MAPPING

Route visualization, warehouse markers

ARCHITECTURE GUARANTEES

<50ms

CHECKOUT LATENCY

Deterministic routing target

ACID

TRANSACTION SAFETY

PostgreSQL + Redis locks

0%

AI DEPENDENCY

Checkout never waits for AI

Per-SKU

LOCK GRANULARITY

No coarse warehouse locks

Hybrid Dual-Path Architecture

● SYNCHRONOUS PATH

Member 1 (Routing) + Member 2 (Checkout)

1. Customer order received
2. Routing engine evaluates warehouses [M1]
3. Cost function scores options [M1]
4. Bin-packing determines packaging [M1]
5. Redis lock + PostgreSQL transaction [M2]
6. Instant order confirmation [M2]

Target: <50ms · Zero AI dependency

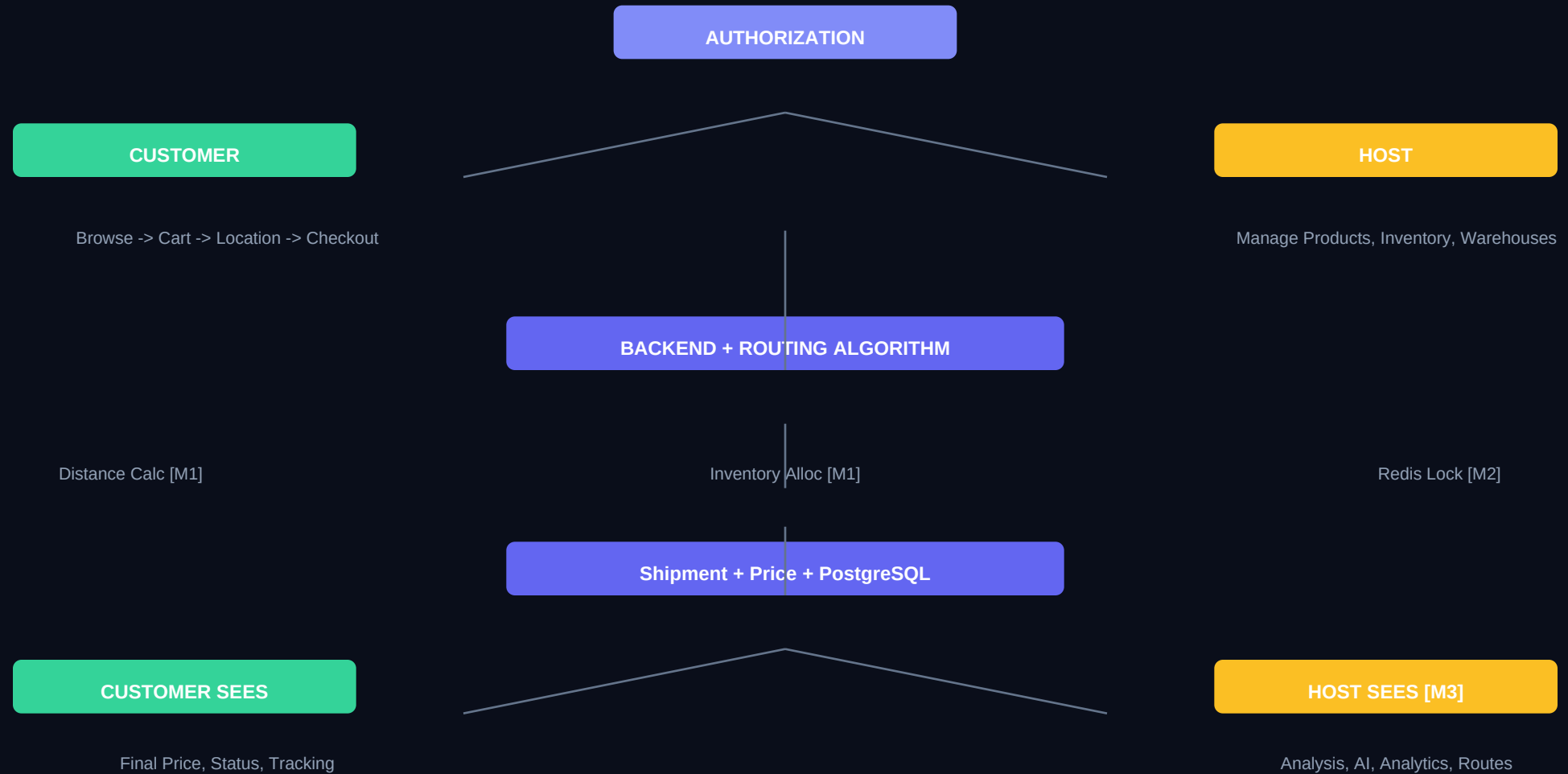
● ASYNCHRONOUS PATH

Member 2 (Gemini) + Member 3 (UI)

1. Checkout renders immediately
2. Frontend requests explanation [M3]
3. Backend forwards to Gemini [M2]
4. AI generates explanation [M2]
5. Fallback if Gemini unavailable [M2]
6. Explanation displayed to host [M3]

Non-blocking · Decoupled from checkout

End-to-End System Flow



Multi-Warehouse Routing Algorithm

Member 1 — Core Algorithms, Database & Frontend Foundation Lead

STEP 1

Find Active WH

Query PostgreSQL

STEP 2

Check SKU Avail

avail > reserved

STEP 3

Haversine Dist

Distance to customer

STEP 4

Score WHs

Cost function scoring

STEP 5

Multi-WH Alloc

Split if needed

STEP 6

Gen Shipments

Create + calc cost

MULTI-WAREHOUSE ALLOCATION EXAMPLE

Required: 10 units · Customer: Jaipur (26.91°N, 75.78°E)

Warehouse A (Mumbai, 8.2 km) -> Avail: 4 units -> Allocate: 4

Warehouse B (Bangalore, 12.5 km) -> Avail: 6 units -> Allocate: 6

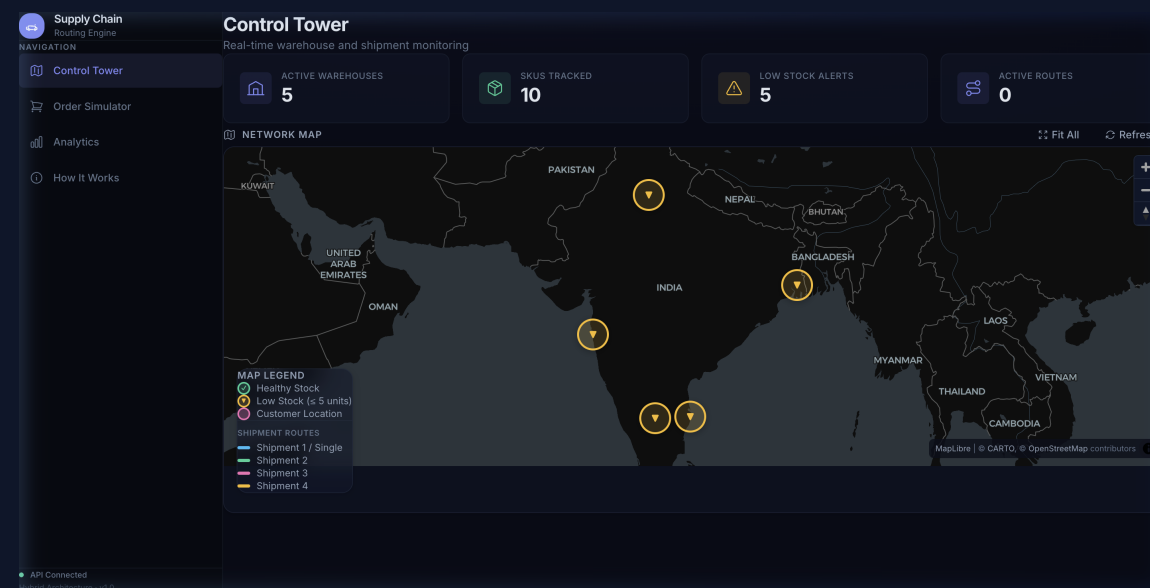
Result: 2 separate shipments · Total distance: 20.7 km

Database Schema & Control Tower UI Foundation

DATABASE SCHEMA (PostgreSQL)

users - Accounts, roles
products - SKU, name, price, category
warehouses - Name, city, lat/lng, capacity
warehouse_inventory - Per-WH per-SKU stock
orders - Customer, status, total, address
order_items - Order -> Product -> qty -> price
shipments - Order -> Warehouse -> tracking
shipment_items - Shipment -> SKU -> quantity
routes - Order -> GeoJSON, distance, cost
inventory_movements - Audit trail: type, qty, ts

CONTROL TOWER UI FOUNDATION



Enterprise dark navy design system · MapLibre GL · KPI cards

Sidebar navigation · API health · Reusable components

Checkout Orchestration & Redis Concurrency

Member 2 — API, Logistics Orchestration & Checkout Lead

CHECKOUT FLOW (SINGLE ACID TRANSACTION)

1. Validate Customer
2. Validate Cart
3. Read Current Inventory
4. Acquire Redis Lock (per-SKU)
5. Check Availability
6. Run Routing Algorithm [calls M1]
7. Allocate Inventory (-avail, +reserved)
8. Create Order + Order Items
9. Create Shipments + Shipment Items
10. Generate Route GeoJSON
11. Calculate Delivery Cost
12. COMMIT Transaction

ON FAILURE: ROLLBACK -> Release Locks -> Error

REDIS CONCURRENCY PROTECTION

Per-SKU distributed locks: lock:sku:<ID>

Lock TTL prevents deadlocks

Rate limiting: 30 req/min per IP

Duplicate checkout prevention

Request validation & input guards

GUARANTEES:

Zero overselling

Zero negative inventory

Zero duplicate orders

BUILT-IN STRESS TEST:

Flash-sale fires N concurrent
checkout requests to prove safety

Gemini AI Explainability & Order Simulator

GEMINI AI INTEGRATION

Google Gemini API: plain-language
routing explanations
Fully async — never blocks checkout
Host-only (customer never sees)

CIRCUIT BREAKER PATTERN:

5 failures -> circuit OPENS
Deterministic fallback activates
60s cooldown, then retry (HALF_OPEN)

AI EXPLAINS:

- Why a warehouse was selected
- Why an order was split
- Distance vs inventory tradeoffs

CUSTOMER ORDER SIMULATOR (Week 5)

```

15 16 # Google Maps Distance Matrix API key (Member 2 – optional)
17 GOOGLE_MAPS_API_KEY=
18
19 # Google Gemini API key (Member 2 – async AI explainability)
20 GEMINI_API_KEY=
21
22 ————— Rate Limiting (Week 4) —————
23 # Sliding window rate limiter (milli
24 RATE_LIMIT_WINDOW=
25 # Max checkout request window (default: 30)
26 RATE_LIMIT_MAX=30
27
28 ————— Gemini Circuit Breaker (Week 4) —————
29 # Consecutive failures before tripping circuit to OPEN (de
30 GEMINI_FAILURE_THRESHOLD=5
31 # Time in ms to keep circuit OPEN before testing HALF_OPEN
32 GEMINI_COOLDOWN_MS=60000
33

```

[Revert this change](#)

ems Output **Terminal** ... zsh + v @ ... | ×

8 cities + custom coords · 10 products · Cart · Full checkout flow

Control Tower Frontend Integration

Member 3 — Frontend, Analytics & Geospatial Visualization Lead

MapLibre route visualization
Warehouse markers (health-coded)
Click marker -> inventory popup
Shipment route lines (color-coded)

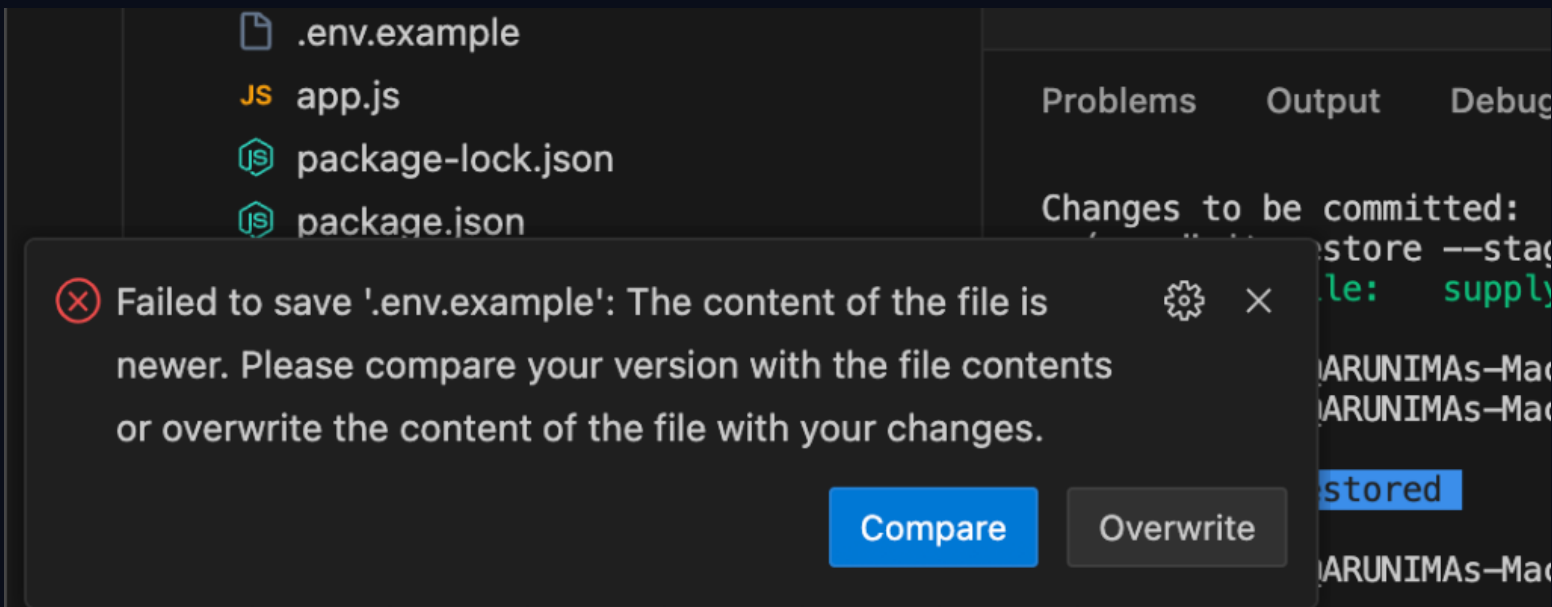
POPOP DETAILS:

Warehouse name & coordinates
Stock status badge (Healthy/Low)
SKU-level Available / Reserved

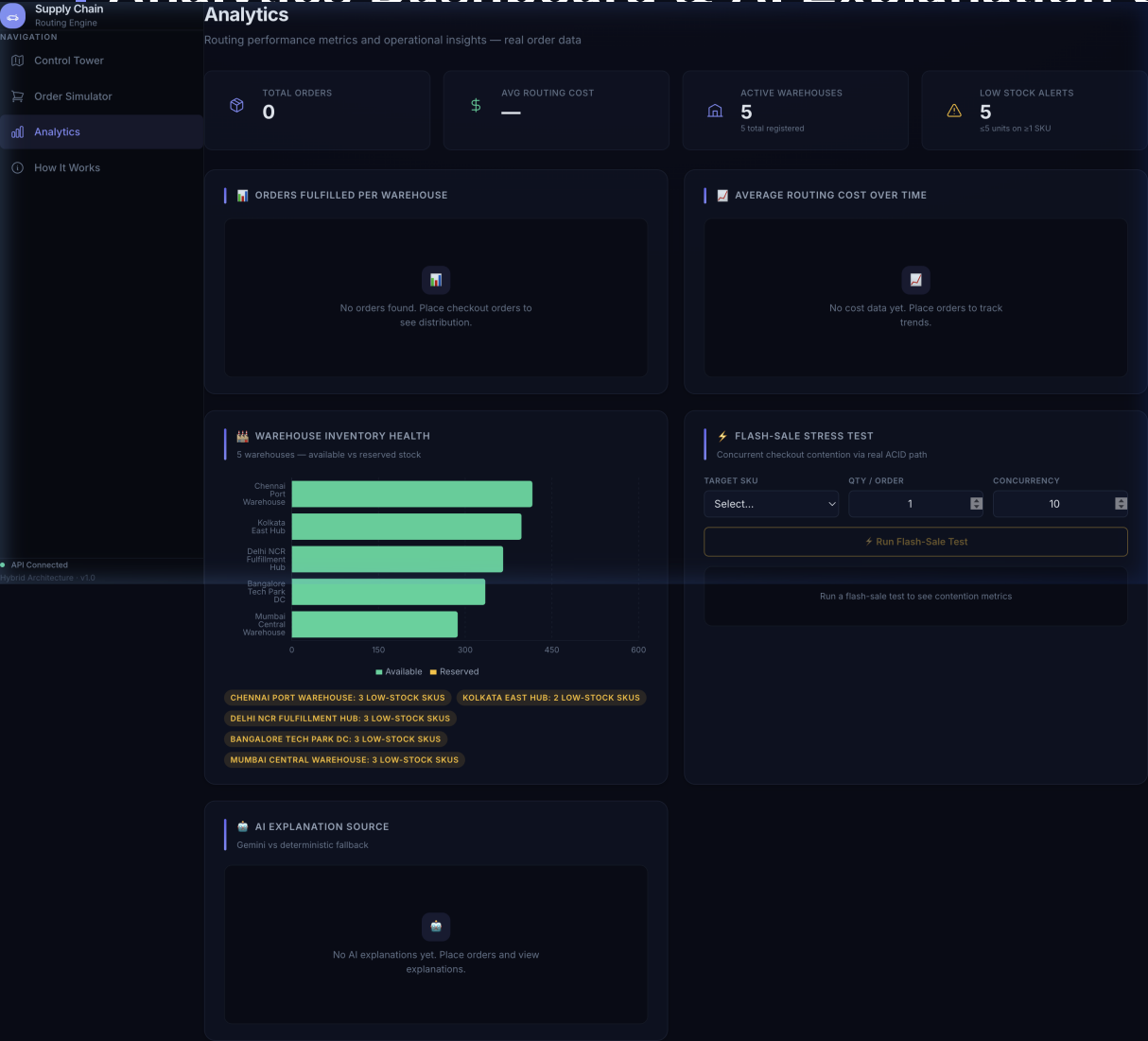
5 WAREHOUSES:

Mumbai Central
Bangalore Tech Park DC
Chennai Port
Kolkata East Hub
Delhi NCR Fulfillment Hub

Frontend API integration
Responsive design & polish

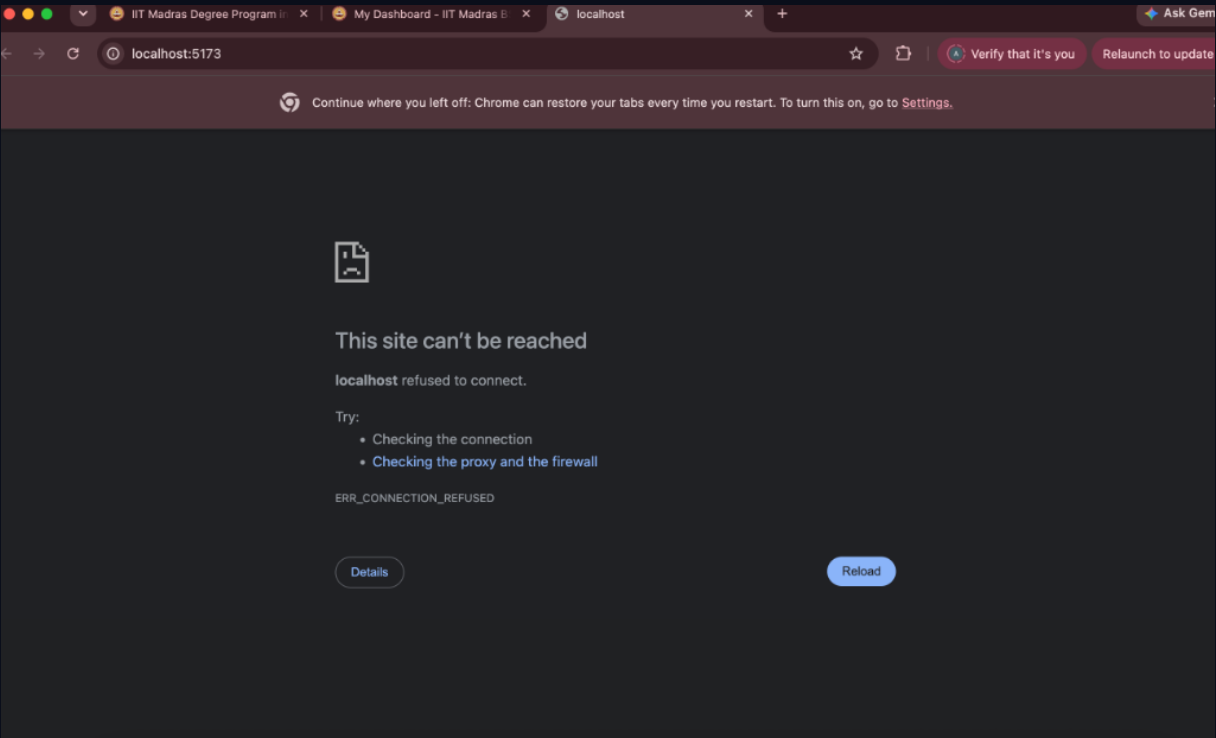


Analytics Dashboard & AI Explanation UI



ANALYTICS DASHBOARD (Recharts)

KPIs · Orders per WH · Routing cost · Inventory health · Low-stock badges



AI EXPLANATION UI

Gemini vs deterministic · AI source breakdown · Flash-sale stress test

How It Works — Interactive Architecture Guide

Supply Chain
Routing Engine

NAVIGATION

Control Tower

Order Simulator

Analytics

How It Works

Bin-Packing

Items packed into smallest box (S → M → L). If items exceed one box, order splits across warehouses.

STEP 05 SYNC

Inventory Protection

Per-SKU Redis lock prevents overselling. PostgreSQL ACID transaction reserves stock atomically.

STEP 06 SYNC

Instant Confirmation

Routing result, cost breakdown, and shipment details returned. No AI involved. Target: <50ms.

ASYNC BOUNDARY — AFTER CHECKOUT RENDERS

STEP 07 ASYNC

AI Explanation

Separately, Gemini AI receives routing data and generates a plain-language explanation. Non-blocking.

DUAL-PATH ARCHITECTURE

SYNCHRONOUS PATH

1. Customer order received

2. Routing engine evaluates warehouses

3. Cost function scores options

4. Bin-packing determines packaging

5. Redis lock + PG transaction

6. Instant order confirmation

Target: <50ms - Zero AI dependency

ASYNCHRONOUS PATH

1. Checkout renders immediately

2. Frontend requests explanation

3. Backend forwards to Gemini

4. AI generates explanation

5. Fallback if Gemini unavailable

6. Explanation displayed to user

Non-blocking - Decoupled from checkout

TECHNOLOGY STACK

React 19 + Vite

FRONTEND

Component-driven SPA with hot reload

Node.js + Express

API SERVER

RESTful endpoints, middleware chain

PostgreSQL

DATABASE

ACID transactions, relational integrity

Redis

CACHE / LOCKS

Per-SKU distributed locks, rate limiting

Gemini API

AI ENGINE

Async explanation generation, fallback

MapLibre GL

MAPPING

Route visualization, warehouse markers

ARCHITECTURE GUARANTEES

<50ms

CHECKOUT LATENCY

Deterministic routing target

ACID

TRANSACTION SAFETY

PostgreSQL + Redis locks

0%

AI DEPENDENCY

Checkout never waits for AI

Per-SKU

LOCK GRANULARITY

No coarse warehouse locks

- 7-step visual walkthrough:
1. Customer Order
 2. Warehouse Evaluation [M1]
 3. Cost Scoring [M1]
 4. Bin-Packing [M1]
 5. Inventory Protection [M2]
 6. Instant Confirmation [M2]
 7. AI Explanation [M2+M3]
- SYNC/ASYNC step labels
- Async boundary marker
- Dual-path comparison
- Technology stack grid
- Architecture guarantees panel
- Designed for viva demo

Supply Chain
Routing Engine

NAVIGATION

Control Tower

Order Simulator

Analytics

How It Works

Order Simulator

Browse products, build your cart, and experience the hybrid routing engine

Location

Products

Cart

Checkout

Checkout

Flash Sale

Delivery Location

Where should we ship your order?

Mumbai

Bangalore

Chennai

Hyderabad

Kolkata

Pune

Jaipur

Ahmedabad

Custom Coordinates

Product Catalog

10 products available

Mirrorless Camera Bo...

SKU-CAMERA-010

152 · 5 warehouses

+ Add to Cart

100W USB-C Charger

SKU-CHARGER-008

32 · 4 warehouses

+ Add to Cart

Wireless Noise-Canc...

SKU-HEADPH-003

216 · 5 warehouses

+ Add to Cart

Mechanical Keyboard

SKU-KEYBOARD-006

354 · 5 warehouses

+ Add to Cart

UltraBook 14"

SKU-LAPTOP-002

99 · 5 warehouses

+ Add to Cart

27" 4K Monitor

SKU-MONITOR-005

380 · 5 warehouses

+ Add to Cart

Ergonomic Wireless ...

SKU-MOUSE-007

212 · 4 warehouses

+ Add to Cart

Smartphone Pro 15

SKU-PHONE-001

6 · 3 warehouses

+ Add to Cart

Portable Bluetooth S...

SKU-SPEAKER-009

47 · 4 warehouses

+ Add to Cart

Digital Tablet 11"

SKU-TABLET-004

315 · 5 warehouses

+ Add to Cart

Your Cart

Your cart is empty

Browse products and add items to get started

Place Order

API Connected

Hybrid Architecture - v1.0

CUSTOMER SEES:

- 8 preset delivery cities
- Custom lat/lng entry
- 10 products with live stock
- Color-coded availability bars
- Cart with quantity controls
- Final price (includes delivery)
- Order confirmation

CUSTOMER DOES NOT SEE:

- Warehouse selection logic
- Routing algorithm scores
- AI explanation
- Inventory locks
- System analytics

PRINCIPLE:

- Customer sees results.
- Host sees analysis.

Live Demo & Test Results

DEMO FLOW

1. Control Tower — 5 warehouses on map [M1+M3]
2. Click warehouse -> inventory popup [M3]
3. Order Simulator — select Jaipur [M2]
4. Place Order -> routing runs [M1+M2]
5. Control Tower — new route [M3]
6. Analytics — updated charts [M3]
7. Flash-Sale — 10 concurrent -> 0 oversell [M2]

TEST RESULTS

Single-warehouse checkout	■ Pass
Multi-warehouse split	■ Pass
Insufficient stock rejection	■ Pass
Concurrent checkout (10 threads)	■ 0 overselling
Gemini AI explanation	■ Generated
Gemini fallback (circuit breaker)	■ Deterministic fallback
Rate limiting (30 req/min)	■ Enforced

```
greSQL connection string
SE_URL=postgresql://postgres:YOUR_PASSWORD@localhost
SE_URL=postgres://arunimasharma@localhost:5432/suppl

s connection string (used by Member 2 for distribute
URL=redis://localhost:6379
URL=

le Maps Distance Matrix API key (Member 2 – optional
E_MAPS_API_KEY=

le Gemini API key (Member 2 – async AI explainabilit
_API_KEY=
```

Copy changed line
Revert this change

Conclusion & Future Scope

WHAT WE BUILT

Multi-warehouse routing algorithm [M1]
Split shipments & bin-packing [M1]
PostgreSQL schema & performance [M1]
Control Tower UI foundation [M1]
Backend APIs & checkout orchestration [M2]
Redis concurrency protection [M2]
Gemini AI + circuit breaker [M2]
Order Simulator frontend [M2]
Analytics dashboards (Recharts) [M3]
MapLibre route visualization [M3]
AI explanation UI [M3]
Responsive design & final polish [M3]

FUTURE SCOPE

Customer authentication (JWT)
Host warehouse/product CRUD panels
Real-time order tracking (WebSocket)
Google Maps Distance Matrix API
ML-based demand forecasting
Delivery partner assignment
Mobile-responsive customer app
Admin role management
Email/SMS notifications

Thank You